

Laboratory Slides 2

The diffusion chain: a diffusion model is a chain factor graph

DSAA 4011 | Fall 2026-27

Yuwen Huang | Week 2 laboratory

Preparation edition

Overview

- Preface
- Recap: message passing on a chain
- Part A: the diffusion chain (a diffusion model is a chain factor graph)
- Part A, research track: the factorized reverse transition (optional)
- Summary and open questions
- References

Preface

A few remarks about these laboratory slides:

- These slides are used in the Week 2 laboratory of DSAA 4011 (Fall 2026–27). In the session the instructor presents the main ideas and you read the slides. The notebook `lab.ipynb` and the program `chains.py` are for exploration after the session.
- There are two versions of these slides: a version with yellowish boxes () that need to be filled out, and a version in which the boxes have been filled out with a possible solution.
- Every block labelled **Task *n*:** names a cell of the notebook, says what to do there, and says which output you should see. The tasks are numbered 1 to 8 and take about an hour.

Preface

A few remarks about these laboratory slides (continued from the previous slide):

- Every number on these slides is printed by the command `python chains.py`. The command `python chains.py --check` compares every number with brute-force enumeration.
- The section marked “optional” is a research track. It goes beyond the syllabus; do it after the other tasks.
- Citations are written as [\[n\]](#) and are listed in full, with all authors, the title and the publication venue, in the section “References”.

Preface

A few remarks about these laboratory slides (continued from the previous slide):

- The terminology follows the papers that are cited. The words **forward process**, **reverse process**, **transition matrix** and **noise schedule** are those of [1], [2] and [5]. Every such word is also defined on the slide where it first appears.
- Definitions, lemmas, claims and examples are numbered within each part. “Lecture Slides 2” refers to the lecture deck of this week.
- If you see an error or a typo, please send an email to `yuwenhuang@hkust-gz.edu.cn`.

Recap: Message Passing on a Chain

Definition: Let $L \in \mathbb{Z}_{>0}$. A **chain factor graph** has the variables $X_1, \dots, X_L$, one local function $u_t(x_t)$ of degree one at every variable X_t , and one local function $f_t(x_{t-1}, x_t)$ of degree two between every pair of neighbouring variables. Its global function is

$$g(x_1, \dots, x_L) \triangleq \prod_{t=1}^L u_t(x_t) \prod_{t=2}^L f_t(x_{t-1}, x_t).$$

Note: A chain has no cycle. Therefore Theorem 1 of Lecture Slides 2 applies. **On a chain, the sum-product algorithm computes every marginal of g exactly.** Theorem 2 of Lecture Slides 2 says the same for the max-product algorithm: it finds the assignment of largest weight.

Recap: Message Passing on a Chain

Lemma 1: (Theorem 1 of Lecture Slides 2, on a chain.) The two update rules

$$m_{X_t \rightarrow f_{t+1}}(x_t) = u_t(x_t) m_{f_t \rightarrow X_t}(x_t) \quad (\text{variable rule}) \quad (1)$$

$$m_{f_{t+1} \rightarrow X_{t+1}}(x_{t+1}) = \sum_{x_t} f_{t+1}(x_t, x_{t+1}) m_{X_t \rightarrow f_{t+1}}(x_t) \quad (\text{function rule}) \quad (2)$$

are applied twice: once from left to right (the **forward sweep**) and once from right to left (the **backward sweep**). **The marginal of g at X_t is then the product of u_t and the two messages that arrive at X_t , normalized.**

Note: If the sum in (2) is replaced by a maximum, the same two sweeps compute the assignment of largest weight. We call the two versions the **sum-product** and the **max-product** algorithm.

Recap: Message Passing on a Chain

Note: In the file `chains.py`, the two sweeps of Lemma 1 are the function `chain_messages(unary, pairwise, mode)`. The argument `unary` is the list of the tables u_t , the argument `pairwise` is the list of the tables f_t , and `mode` is "sum" or "max". The function returns the messages and the beliefs. It also returns the total: the partition function Z for "sum", the largest weight for "max". For "max" it returns the assignment that attains the largest weight as well.

Note: The function `enumerate_chain` computes the same quantities by listing every assignment of the chain. It shares no code with `chain_messages`. It is the reference. **A number from message passing is trusted only after enumeration has confirmed it.**

Recap: Message Passing on a Chain

Task 1: Run the first cell of the notebook. Then open `chains.py` and read `chain_messages`, which has thirty lines. Find the line that implements rule (1) and the line that implements rule (2).

Expected output: none; this task is reading.

Note: The tasks of the notebook are numbered as on these slides. Task 5 is an exploration: you choose one question from a list, write a hypothesis and a prediction, and then run one cell. The other tasks are fixed. If a task takes much longer than the others, move on; nothing later depends on finishing it.

Part A: The Diffusion Chain

Definition 1: A **picture** has two **pixels**. Every pixel is either dark, written “.”, or bright, written “#”. Therefore there are four pictures, which we always list in the order

.., .#, #., ##.

Definition 2: The **data distribution** $q(x_0)$ is the probability distribution of a clean picture X_0 . In this laboratory,

$$q(x_0) = (0.55, 0.05, 0.05, 0.35)$$

over the four pictures in the order above. In words: most clean pictures are all-dark or all-bright, and the two mixed pictures are rare.

Part A: The Diffusion Chain

Definition 3: Let $T = 4$. The **forward process** produces the pictures $X_1, \dots, X_T$ from X_0 in T **noise steps**. In noise step t , each pixel of X_{t-1} flips with probability β_t . The two pixels flip independently. The result is X_t . The sequence $(\beta_1, \dots, \beta_T) = (0.1, 0.2, 0.3, 0.4)$ is the **noise schedule**.

Definition 4: The **transition matrix** Q_t of noise step t is the 4×4 matrix with entries

$$Q_t[a, b] \triangleq q(x_t = b \mid x_{t-1} = a),$$

In words: $Q_t[a, b]$ is the probability that picture a becomes picture b in noise step t . The rows of Q_t are indexed by x_{t-1} , the columns by x_t . Every row sums to one.

Part A: The Diffusion Chain

Example 1: For $\beta_1 = 0.1$, a pixel keeps its value with probability 0.9 and flips with probability 0.1. Since the two pixels flip independently,

$$Q_1 = \begin{pmatrix} 0.81 & 0.09 & 0.09 & 0.01 \\ 0.09 & 0.81 & 0.01 & 0.09 \\ 0.09 & 0.01 & 0.81 & 0.09 \\ 0.01 & 0.09 & 0.09 & 0.81 \end{pmatrix}.$$

For instance, $Q_1[. ., \# \#] = 0.1 \cdot 0.1 = 0.01$, because both pixels must flip. In `chains.py` this matrix is `transition_matrix(0.1)`.

Part A: The Diffusion Chain

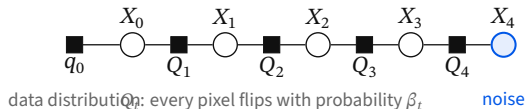
Note: The same construction with a matrix Q_t over a vocabulary of tokens is the discrete forward process of [2]. With real-valued pixels and Gaussian noise in place of flips, it is the forward process of [1]. **In both cases every position is corrupted independently of the other positions.**

Claim 1: **The forward process is a chain factor graph.** Its global function is the joint distribution of $(X_0, X_1, \dots, X_T)$, namely

$$q(x_0, x_1, \dots, x_T) = q(x_0) \prod_{t=1}^T Q_t[x_{t-1}, x_t]. \quad (3)$$

The graph is drawn on the next slide.

Part A: The Diffusion Chain



Proof of Claim 1: The forward process is a Markov chain. Its initial distribution is $q(x_0)$ and its transition matrices are $Q_1, \dots, Q_T$. The joint distribution of a Markov chain is the product of its initial distribution and its transition probabilities. This is (3). In the words of Lecture Slides 2: $q(x_0)$ is the local function of degree one at X_0 , and Q_t is the local function of degree two between X_{t-1} and X_t . $\square$

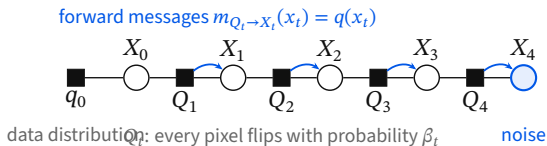
Part A: The Diffusion Chain

Lemma 2: The forward messages are the marginals of the forward process. For every $t \in \{1, \dots, T\}$,

$$m_{Q_t \rightarrow X_t}(x_t) = q(x_t) = \sum_{x_{t-1}} q(x_{t-1}) Q_t[x_{t-1}, x_t]. \quad (4)$$

Proof: The variables $X_1, \dots, X_T$ carry no local function of degree one. Hence rule (1) passes every message on unchanged: $m_{X_{t-1} \rightarrow Q_t} = m_{Q_{t-1} \rightarrow X_{t-1}}$ for $t \geq 2$, and $m_{X_0 \rightarrow Q_1} = q(x_0)$. Rule (2) then gives (4), by induction on t . $\square$

Part A: The Diffusion Chain



Note: The figure shows the forward messages of Lemma 2. In [1], $q(x_t)$ is called the marginal of the forward process at step t . It is determined by the data distribution and the noise schedule. **The noise schedule of a diffusion model is a sequence of forward messages.** In `chains.py`, the list $q(x_0), \dots, q(x_T)$ is `forward_marginals()`.

Part A: The Diffusion Chain

Question: What does $q(x_4)$ look like, and why?

Task 2: In the notebook cell of Task 2, write your answer to the question above. Then run the cell that prints $q(x_0), \dots, q(x_4)$ by the forward sweep. Expected output: five lines, from $q(x_0) = (0.55, 0.05, 0.05, 0.35)$ to $q(x_4) \approx (0.254, 0.250, 0.250, 0.247)$.

Part A: The Diffusion Chain

Note: $q(x_4)$ is not exactly uniform. With more noise steps, or with larger β_t , it comes closer to uniform. Exploration E1 of Task 5 asks which noise schedules reach uniform, and after how many steps.

Definition 5: The **reverse process** runs the chain from X_T back to X_0 . Its transition of step t is the conditional distribution $p(x_{t-1} | x_t)$. We call it the **reverse transition** of step t . We write it as the matrix R_t with $R_t[b, a] \triangleq p(x_{t-1} = a | x_t = b)$. The rows are indexed by the noisy picture x_t . The columns are indexed by the cleaner picture x_{t-1} . Every row sums to one.

Part A: The Diffusion Chain

Lemma 3: The reverse transition is a ratio of forward messages. For every $t \in \{1, \dots, T\}$,

$$p(x_{t-1} | x_t) = \frac{Q_t[x_{t-1}, x_t] q(x_{t-1})}{q(x_t)}. \quad (5)$$

Proof: By Bayes' rule, $p(x_{t-1} | x_t) = q(x_{t-1}, x_t)/q(x_t)$. By (3), $q(x_{t-1}, x_t) = q(x_{t-1}) Q_t[x_{t-1}, x_t]$. By Lemma 2, the factor $q(x_{t-1})$ is the forward message at X_{t-1} , and the denominator $q(x_t)$ is the forward message at X_t . $\square$

Note: A diffusion model [1,2,5] is a neural network trained to approximate the reverse transitions (5). On our chain there are only four pictures, so the reverse transitions can be computed exactly. That is Task 3.

Part A: The Diffusion Chain

Example 2: The reverse transition of step 4, given the noisy picture $x_4 = \cdot \cdot$, is the row

$$R_4[\cdot \cdot, \cdot] = (0.392, 0.229, 0.229, 0.150)$$

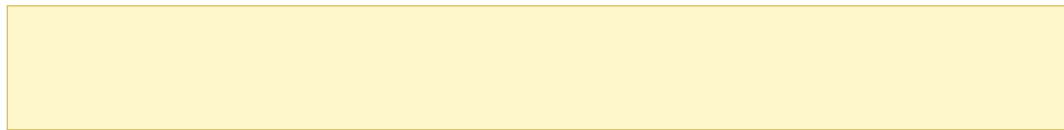
over $x_3 \in \{\cdot \cdot, \cdot \#, \# \cdot, \#\#\}$. In words: an all-dark noisy picture most probably came from an all-dark picture. With probability 0.150 it came from an all-bright picture.

Task 3: Write `my_reverse_transition(t)` in the notebook, using (5). First form the joint table with entries $Q_t[a, b] q(x_{t-1} = a)$. Then divide every column by its sum. Then transpose, so that the rows are indexed by x_t . The cell compares your table with `chains.reverse_transition`.

Expected output: four lines, one per step, each ending in `agrees with the reference: True`.

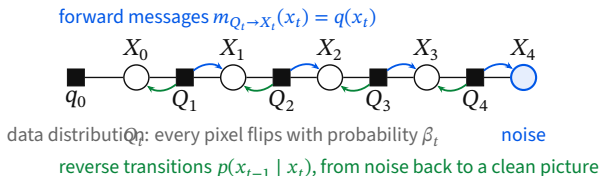
Part A: The Diffusion Chain

Question: Which forward message appears in the numerator of (5), and which in the denominator?



Definition 6: Ancestral sampling of the reverse process draws x_T from $q(x_T)$, then x_{T-1} from $p(x_{T-1} | x_T)$, and so on, down to x_0 from $p(x_0 | x_1)$. Each draw uses one row of one reverse transition. It is the row of R_t for the picture x_t drawn just before.

Part A: The Diffusion Chain



Claim 2: Ancestral sampling with the exact reverse transitions reproduces the data distribution. That is, the clean picture x_0 it produces is distributed according to $q(x_0)$.

Proof: By the chain rule of probability, $q(x_T) \prod_{t=1}^T p(x_{t-1} | x_t) = q(x_0, \dots, x_T)$. The marginal distribution of X_0 under the right-hand side is $q(x_0)$. $\square$

Note: This is how a trained diffusion model generates a picture [1]. It starts from noise. Then it applies its learned reverse transitions, one step at a time.

Part A: The Diffusion Chain

Note: In `chains.py` the sampler of Definition 6 is `ancestral_sample(n, rng, reverse_fn)`, where `reverse_fn(t)` returns the matrix to be used as R_t ; by default the exact one is used.

Task 4: Write down the proportions of the four pictures that you expect from ancestral sampling. Then run the cell that draws 5000 pictures with your own `my_reverse_transition`.

Expected output: proportions close to the data distribution, for example (0.543, 0.054, 0.043, 0.359) against (0.55, 0.05, 0.05, 0.35).

Part A: The Diffusion Chain

Question: Suppose that the forward message $q(x_{t-1})$ is dropped from the numerator of (5), so that the reverse transition becomes the transition matrix run backwards. What does ancestral sampling produce then?

Definition 7: The **denoising posterior** is the conditional distribution $p(x_0 \mid x_t = b)$ of the clean picture, given that the picture b was observed at noise step t . On the chain factor graph it is a belief. Attach the evidence $[x_t = b]$ to X_t as a local function of degree one (Definition 3 of Lecture Slides 2). **The denoising posterior is then the belief at X_0 .**

Part A: The Diffusion Chain

Example 3: For $b = \#.$ observed at step $t = 4$, the two sweeps give

$$p(x_0 \mid x_4 = \#.) = (0.5498, 0.0463, 0.0540, 0.3499),$$

and the enumeration of all $4^4 = 256$ completions of the chain gives the same four numbers.

Task 6: Run the cell that computes the posterior of Example 3 by message passing (`chains.denoising_posterior`) and by enumeration (`starter.enumerate_marginals`).

Expected output: the two lines of Example 3, then agree: True.

Note: In the notebook, Task 5 comes before Task 6.

Part A: The Diffusion Chain

Task 5: Choose one of the explorations E1 to E5 on the next slide (more if you wish). Write a hypothesis and a prediction in the notebook before you run anything. Then run. Then write one sentence about what you found. If you are unsure which to choose, take E5: its cell is already written.

Expected output: whatever you predicted, or something else; both are results.

Note: Every helper of `chains.py` accepts your own data distribution, noise schedule and number of pixels. So every exploration can be run with the functions that the tasks already used.

Part A: The Diffusion Chain

The explorations of Task 5.

- E1 Noise schedules.** Which schedules bring $q(x_T)$ within 0.01 of uniform, and after how many steps? Use `chains.steps_until_noise`.
- E2 Observing earlier.** Observe the same noisy picture at $t = 1, 2, 3, 4$. How does the denoising posterior change with t ?
- E3 Changing the data.** For which data distributions is R_t the transition matrix run backwards? Predict, then test with `np.allclose`.
- E4 More pixels.** With `pixels=3` there are eight pictures. How many entries does R_t have, and how does that number grow?
- E5 Breaking it on purpose.** Drop $q(x_{t-1})$ from (5), predict what ancestral sampling produces, then run the cell.

Part A: The Diffusion Chain

Note: (What Part A has shown.)

- **A diffusion model is a chain factor graph** (Claim 1).
- **Its noise schedule is a sequence of forward messages** (Lemma 2).
- **The reverse transition it learns is a ratio of forward messages** (Lemma 3).
- **Sampling from it is ancestral sampling on the chain** (Claim 2).
- **Denoising an observed picture is the belief at X_0** (Definition 7).

Note: A real diffusion model does the same with millions of pixels. A neural network takes the place of R_t . The research track asks what that network can and cannot represent.

Part A, Research Track (optional)

Question: A picture with n pixels has 2^n possible values. How many entries does the reverse transition R_t have?

Answer: R_t has one row per value of x_t and one column per value of x_{t-1} , hence $(2^n)^2 = 4^n$ entries:

pixels n	pictures 2^n	entries of R_t , 4^n
2	4	16
3	8	64
10	1024	1 048 576
10^6	2^{10^6}	4^{10^6}

Note: A real pixel has 256 grey levels. A position in a text has a vocabulary of tens of thousands of tokens. So the true numbers are larger still. **No network can output R_t , and no memory can store it.**

Part A, Research Track (optional)

What diffusion models output instead of R_t : one distribution per position.

Note: (Image diffusion.) Equation (1) of [1], with the covariance fixed to $\sigma_t^2 I$, is

$$p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(\mu_{\theta}(x_t, t), \sigma_t^2 I).$$

The covariance matrix is a multiple of the identity I . **Given x_t , the pixels of x_{t-1} are drawn independently of each other.** The network outputs the mean $\mu_{\theta}(x_t, t)$, one number per pixel. It outputs nothing that couples two pixels.

Part A, Research Track (optional)

Note: (Discrete diffusion.) Section 3 of [2], and the masked diffusion language models [3,4] built on it, use

$$p_{\theta}(x_{t-1} \mid x_t) = \prod_i p_{\theta}(x_{t-1}^i \mid x_t),$$

where x^i is position i of x . **There is one categorical distribution per position, and they are multiplied together.**

Definition 8: Let R_t be the exact reverse transition. For a noisy picture b and a pixel k , let $\pi_t^k(b)$ be the probability, under the row $R_t[b, \cdot]$, that pixel k of x_{t-1} is bright. The **factorized reverse transition** of step t is the matrix F_t defined as follows. Row b of F_t assigns to picture a a product of one factor per pixel k : the factor is $\pi_t^k(b)$ if pixel k of a is bright, and $1 - \pi_t^k(b)$ otherwise.

Part A, Research Track (optional)

Note: In words, F_t keeps the per-pixel marginals of R_t and multiplies them, as if the pixels of x_{t-1} were independent given x_t . This is exactly what the networks on the previous two slides represent. In `chains.py` it is `factorized_reverse_transition(t)`.

Example 4: Take the row of Example 2, step 4 given $x_4 = \dots$:
 $R_4[\dots, \cdot] = (0.392, 0.229, 0.229, 0.150)$. Pixel 1 of x_3 is bright in the pictures # . and ##, so $\pi_4^1(\dots) = 0.229 + 0.150 = 0.379$; by symmetry, $\pi_4^2(\dots) = 0.379$ as well. The factorized row is

$$\begin{aligned} F_4[\dots, \cdot] &= (0.621 \cdot 0.621, 0.621 \cdot 0.379, 0.379 \cdot 0.621, 0.379 \cdot 0.379) \\ &= (0.386, 0.235, 0.235, 0.144), \end{aligned}$$

which is close to the exact row.

Part A, Research Track (optional)

Example 4 (continued): At step 1, given $x_1 = \cdot \cdot$, the exact row is $R_1[\cdot \cdot, \cdot] = (0.973, 0.010, 0.010, 0.008)$ and the factorized row is $F_1[\cdot \cdot, \cdot] = (0.965, 0.017, 0.017, 0.000)$. **The last entry of F_1 is 25 times too small.** Under F_1 , an all-dark x_1 almost never came from an all-bright x_0 .

Task 7: Write `my_factorized(t)` from your `my_reverse_transition(t)`. For every row b and every pixel k , sum the row over the pictures in which pixel k is bright. This gives $\pi_t^k(b)$. Then multiply the per-pixel probabilities as in Definition 8. The cell compares your table with `chains.factorized_reverse_transition`.
Expected output: four lines ending in True. (Research track.)

Part A, Research Track (optional)

Question: Why is the factorization justified at all?

Answer: Section 2.2 of [5], citing [6], gives the reason. For a diffusion process whose steps are small, the reverse transition has the same functional form as the forward transition. Our forward transition flips the pixels independently; it is itself a product over the pixels. **Therefore, for small β_t , the exact R_t is close to factorized.** The factorization then loses little. **This is why diffusion models take many small steps.** The independence between the positions inside one step is paid for with the number of steps.

Definition 9: The **factorization error** of step t is $\max_{a,b} |R_t[b, a] - F_t[b, a]|$. It is the largest absolute difference between R_t and F_t , over all entries. In `chains.py` it is `factorization_error(t)`.

Part A, Research Track (optional)

Claim 3: (Checked numerically.) On our noise schedule the factorization error is

step t	1 ($\beta_1 = 0.1$)	2	3	4 ($\beta_4 = 0.4$)	1, with every $\beta_t = 0.02$
factorization error	0.103	0.075	0.034	0.007	0.017

Note: Small steps do make F_t nearly exact, as [5] says. With every $\beta_t = 0.02$, the error at step 1 falls from 0.103 to 0.017. **But on our schedule the smallest step, step 1, has the largest error.** The reason: step 1 is next to the data. The forward message $q(x_0)$ couples the two pixels strongly, and F_t drops exactly that coupling.

Note: Measure “nearly exact” by the absolute difference, not by the ratio of entries. In Example 4 the ratio of two tiny entries is 25, while the entries agree to three decimal places.

Part A, Research Track (optional)

Definition 10: Let $R'_t, t = 1, \dots, T$, be any reverse transitions (exact, factorized, or corrected). Draw $N = 100\,000$ pictures by ancestral sampling with R'_t in place of R_t . Let $\hat{p}(x)$ be the fraction of the draws that equal the picture x . The **sampling error** of R'_t is

$$\max_x |\hat{p}(x) - q(x_0 = x)|,$$

the largest difference, over the four pictures, between the proportion generated and the data distribution. In `chains.py` it is `sampling_error(reverse_fn=...)`. **By Claim 2, the sampling error of the exact R_t is zero**, up to the randomness of the N draws.

Part A, Research Track (optional)

Definition 11: The **pairwise correction** is the vector

$$\varphi(x) \triangleq \frac{q(x_0 = x)}{\prod_k q(x_0^k = x^k)} = (1.53, 0.21, 0.21, 2.19),$$

the ratio of the data distribution to the product of its per-pixel marginals. Here $q(x_0^k = x^k)$ is the probability that pixel k of a clean picture has the value x^k . φ equals one everywhere if and only if the two pixels are independent under $q(x_0)$.

Note: “ φ multiplied in at step t ” means that the row $F_t[b, \cdot]$ is replaced by $F_t[b, x] \varphi(x)$, renormalized to sum to one. “ φ^λ ” means that every entry of φ is raised to the power λ first; $\lambda < 1$ weakens the correction.

Part A, Research Track (optional)

Example 5: (The sampling error of several reverse transitions.)

reverse transition used	proportions $\hat{p}$	sampling error
exact, R_t	0.548, 0.051, 0.051, 0.350	0.002
factorized, F_t	0.493, 0.116, 0.116, 0.275	0.075
factorized, φ multiplied in at step 1 only	0.580, 0.025, 0.025, 0.370	0.030
factorized, $\varphi^{0.25}$ multiplied in at every step	0.531, 0.058, 0.059, 0.352	0.019
factorized, φ multiplied in at every step	0.483, 0.005, 0.005, 0.507	0.157

Note: Under F_t every per-pixel marginal of the samples is exact (0.39 bright, against 0.40 under the data). **Only the coupling between the pixels is wrong.** So the two mixed pictures are over-represented.

Part A, Research Track (optional)

Note: (Example 5, continued.) **The fixed correction φ has the right form but the wrong strength.** At every step it over-corrects. At step 1 only, it under-corrects. By Claim 3, the right strength depends on t .

Definition 12: Take a noise schedule $(\beta_1, \dots, \beta_T)$. The number $\prod_{t=1}^T (1 - 2\beta_t)$ is the factor by which the forward process reduces the correlation between a pixel of X_0 and the same pixel of X_T . We call it the **total noise** of the schedule. The **equal-step schedule** with k steps is the schedule $(\beta, \dots, \beta)$ of length k with the same total noise as the original schedule. In `chains.py` it is `equal_step_schedule(k)`.

Part A, Research Track (optional)

Example 5 (continued): With the factorized reverse transition and the equal-step schedule of k steps, the sampling error is

k	4	8	16	32	64
sampling error	0.135	0.087	0.047	0.025	0.015

Note: Doubling the number of steps roughly halves the error. The factorization costs a fixed fraction of the error per step. The cost is paid in the number of steps. This is the whole economics of sampling from a diffusion model.

Open question 1: which correction between the positions should a diffusion model apply at each step, and how can it be learned?

Part A, Research Track (optional)

Note: (How research thinking works from here.) Example 5 contains a surprise. **A sampler whose every per-pixel marginal is exact still produces the wrong pictures.** The notebook asks you to make five moves with that surprise. The first two:

1. Ask a question that a number can answer, for example “does doubling the number of steps halve the sampling error?”
2. Write the prediction and a threshold before running anything, for example “ φ counts as a repair if the sampling error falls below 0.02.” A threshold, once written, does not move.

Part A, Research Track (optional)

Note: (How research thinking works from here, continued.) The remaining three moves:

3. Name the baseline and the reference: the factorized transition is the baseline that every repair must beat, and the exact transition is the reference, whose error is known.
4. Look for the boundary: where does a repair stop working, or start to hurt? In Example 5, φ at every step hurts.
5. Report a miss as a miss: a prediction that failed, with the reason, is a result. A threshold changed after seeing the number is not.

A question is a research question when three things hold. Nobody knows the answer. The answer would change what someone builds. An experiment that you can run decides it.

Summary and Open Questions

Note: (What this laboratory has shown.)

- **A diffusion model is a chain factor graph** (Claim 1).
- **Its noise schedule is the sequence of forward messages** (Lemma 2).
- **The reverse transition it learns is a ratio of forward messages** (Lemma 3).
- **Sampling from it is ancestral sampling on the chain** (Claim 2).
- **Denoising an observed picture is the belief at X_0** (Definition 7).

Note: **A real diffusion model learns the factorized version F_t , not R_t** (Definition 8). The factorization error depends on the step (Claim 3). The sampling error of F_t falls as the number of steps grows (Example 5).

Summary and Open Questions

The question that this laboratory raises and does not settle:

The coupling between positions. A diffusion model outputs the positions of x_{t-1} independently, given x_t . The error that this causes depends on the step (Claim 3).

Which correction between the positions should be applied at each step, and can it be learned from the same data as the model?

Note: The question can be attacked first on the chain of this laboratory, where every quantity is exact. Then it can be attacked on a small open model. The notebook ends with a three-rung ladder from the first to the second. The third rung is an acceptable Individual Inquiry Project.

Summary and Open Questions

Task 8: Fill in `evidence_card.md`. For the exploration you chose, record the hypothesis, the prediction, the result and one sentence of explanation. Then write one claim that you are willing to defend, the single test that supports it most strongly, and one condition that you did not test.

Expected output: none; save the card and the notebook. This laboratory is formative practice and carries no separate mark.

Note: (What is examined.) A language model writes the sum-product algorithm for you in under a minute. **The evidence card examines four things.** The hypothesis and the prediction that you wrote before each run. The reference that you compared against. The boundary case that you found. Your explanation of why the chain behaved as it did.

Summary and Open Questions

Note: (Names on the slides and in the code.) The names below are spelled identically on the lecture slides, in `chains.py`, in `starter.py` and in `README.md`.

on the slides	in the code
the two rules (1) and (2), on a chain	<code>chain_messages(unary, pairwise, mode)</code>
the reference, on a chain	<code>enumerate_chain</code>
$m_{f \rightarrow i}$, on any tree	<code>factor_to_variable</code>
$m_{i \rightarrow f}$, on any tree	<code>variable_to_factor</code>
the normalized belief	<code>normalize</code>
the reference, on any tree	<code>enumerate_marginals</code>

The last four rows are the exercises of `starter.py`; they run on the diffusion chain as well.

Summary and Open Questions

Note: (Names on the slides and in the code, continued.)

on the slides	in the code
the transition matrix Q_t	<code>transition_matrix</code>
the marginals $q(x_t)$	<code>forward_marginals</code>
the reverse transition R_t	<code>reverse_transition</code>
ancestral sampling	<code>ancestral_sample</code>
the denoising posterior	<code>denoising_posterior</code>
the factorized reverse transition F_t	<code>factorized_reverse_transition</code>
the factorization error	<code>factorization_error</code>
the sampling error	<code>sampling_error</code>
the pairwise correction φ	<code>pairwise_correction</code>

References

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 6840–6851, 2020. arXiv:2006.11239.
- [2] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, “Structured denoising diffusion models in discrete state-spaces,” in *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, pp. 17981–17993, 2021. arXiv:2107.03006.
- [3] S. S. Sahoo, M. Arriola, Y. Schiff, A. Gokaslan, E. Marroquin, J. T. Chiu, A. Rush, and V. Kuleshov, “Simple and effective masked diffusion language models,” in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. arXiv:2406.07524.
- [4] S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li, “Large language diffusion models,” in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pp. 50608–50646, 2025. arXiv:2502.09992.

References

- [5] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML 2015)*, PMLR vol. 37, pp. 2256–2265, 2015.
- [6] W. Feller, “On the theory of stochastic processes, with particular reference to applications,” in *Proceedings of the [First] Berkeley Symposium on Mathematical Statistics and Probability*, University of California Press, pp. 403–432, 1949.
- [7] F. R. Kschischang, B. J. Frey, and H.-A. Loeliger, “Factor graphs and the sum-product algorithm,” *IEEE Transactions on Information Theory*, vol. 47, no. 2, pp. 498–519, 2001.
- [8] M. Arriola, A. Gokaslan, J. T. Chiu, Z. Yang, Z. Qi, J. Han, S. S. Sahoo, and V. Kuleshov, “Block diffusion: interpolating between autoregressive and diffusion language models,” in *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*, 2025. arXiv:2503.09573.

Note: [7] is the paper that introduced factor graphs and the sum-product algorithm in the form used in Lecture Slides 2. [8] is not cited on the slides; it is the reading for the ladder at the end of the notebook.